

Serenity Shift

A Therapeutic PC Experience for Agoraphobia Exposure Training

Project Book

Submitted by: [Yarin Keshet]
Supervisor: [Erel Segal-Halevi]
[Ariel University - Computer Science]
[August 2026]

Table of Contents

Table of Contents.....	2
1. Abstract.....	4
2. Introduction.....	4
2.1 Background.....	4
2.2 Project Goals.....	5
2.3 Target Audience.....	5
3. Comparable Products / Approaches.....	5
4. System Requirements.....	6
4.1 Functional Requirements.....	6
4.2 Non-Functional Requirements.....	7
5. Architecture & Design.....	7
5.1 System Architecture.....	7
5.2 Technologies Used.....	8
5.3 Scene & Level Structure.....	9
6. Development Process.....	9
6.1 Timeline / Chronological Overview.....	9
6.2 Technical Challenges & Solutions.....	10
7. Key Features.....	11
7.1 Authentication & Progress Saving.....	11
7.2 NPC & Bird-Catching Mechanics.....	11
7.3 The “Django” Puzzle.....	12
7.4 Audio Systems.....	12
7.5 Navigation & Accessibility Aids.....	13
8. Testing & Evaluation.....	13
8.1 Functional Testing.....	13
8.2 Clinical Feedback.....	14
8.3 Public Presentation.....	15
9. Results & Conclusions.....	15
10. Future Work.....	16
11. References.....	16
12. Appendices.....	17
Appendix A: Selected Code Samples.....	17
12.A.1 Asynchronous Scene Loading with a Smoothed Progress Bar.....	17
12.A.2. Cloud-Backed Login with Resume-From-Last-Level.....	18
12.A.3. A Testable Algorithm Extracted from Game Logic (Note-Sequence Puzzle).....	19
12.A.4. Weighted Random Selection Without Replacement.....	19
12.A.5. Centralized Pause State Across Independent Systems.....	20
12.A.6. A Unit Test for the Extracted Puzzle Logic.....	20
12.A.7. NPC Spawning with NavMesh Validation and Obstacle Rejection.....	21
12.A.8. Bird-Catching Mechanic with Multi-Controller Fallback.....	22

12.A.9. Traffic AI: Waypoint Navigation with Sensor-Based Obstacle Avoidance.....	23
Appendix B: Screenshots.....	24
Appendix C: Links.....	26

1. Abstract

Agoraphobia is a form of anxiety disorder in which individuals experience intense fear of open, crowded, or unfamiliar public spaces. Its severity exists on a wide spectrum: for some, it manifests as mild discomfort in specific situations, such as avoiding busy supermarkets or crowded public transport; for others, it escalates into a persistent avoidance of leaving the house altogether, making everyday tasks like grocery shopping, commuting, or even walking outside feel overwhelming. One of the most established treatment approaches for this condition is exposure therapy, in which patients are gradually and safely confronted with anxiety-inducing situations. However, real-world exposure sessions can be difficult to control, costly to arrange, and intimidating for patients to begin.

Serenity Shift addresses this gap by offering a therapeutic PC application that simulates realistic American city environments as a controlled, gradual exposure tool for agoraphobia treatment. The system allows patients to progress through multiple difficulty levels — from tutorial to hard — enabling exposure intensity to be tailored to each patient's needs. Interactive elements such as NPC encounters, a bird-catching mechanic, and an immersive street-performer puzzle ("Django") with spatial 3D audio were designed to keep patients engaged while remaining within a therapeutic framework rather than a purely entertainment-driven one.

The project was built in Unity and includes a full authentication and version-selection system, scene management across difficulty tiers, and a WebGL build optimized for smooth performance across various screens and platforms. The application was deployed to itch.io and presented at Expo TLV and was evaluated through a feedback survey distributed to players to assess its therapeutic potential.

The result is a functional prototype demonstrating that PC-based exposure simulation can offer a scalable, controllable, and engaging alternative to traditional in-person exposure therapy for agoraphobia.

2. Introduction

2.1 Background

Simulated exposure therapy is grounded in the same principles as traditional in-vivo exposure therapy, gradually confronting patients with anxiety-inducing scenarios in a controlled, repeatable environment. A systematic review of the literature on simulation-based exposure therapy found it effective for agoraphobia when delivered across 8 to 12 sessions, typically once a week for at least 15 minutes per session (Frontiers in Psychiatry, 2021).

A key advantage of simulation-based exposure over real-world exposure is controllability: an open public environment in real life is inherently unpredictable — the number of people, ambient noise, and unexpected events cannot be controlled or repeated from session to session. Serenity Shift addresses this by simulating a controlled environment where stress-inducing stimuli can be precisely and consistently adjusted. This includes the number of NPCs present in the scene, the ambient sound of the city (louder and busier at harder difficulty levels versus calmer and quieter in easier ones), and the presence or absence of stressors such as car honking. In addition, the game introduces in-mission tasks designed to redirect the patient's attention away from anxiety-inducing stimuli (attention diversion), a technique that is difficult to reliably reproduce in real-world exposure settings.

2.2 Project Goals

The primary goal of Serenity Shift is to provide a controlled, repeatable exposure environment for individuals experiencing agoraphobia, addressing the limitations of real-world exposure sessions. Specific goals of the project include:

Simulating a realistic public environment — a busy city street — as the setting for gradual exposure exercises. Enabling precise control over stress-inducing stimuli, including the number of NPCs present, the intensity and volume of ambient city sound, and the presence of stressors such as car honking, allowing the level of exposure to be scaled deliberately rather than left to real-world unpredictability.

Structuring exposure into clear difficulty levels (tutorial, easy, medium, hard), so patients can progress gradually as their tolerance builds.

Incorporating in-game missions that promote attention diversion, giving patients a practical technique for redirecting focus away from anxiety-inducing stimuli during exposure.

Delivering the experience on standard PC hardware, making it accessible without the cost or technical barriers of specialized equipment.

Supporting future clinical use by laying the groundwork for therapist-guided sessions and outcome tracking.

2.3 Target Audience

Serenity Shift is designed primarily for individuals diagnosed with agoraphobia, ranging from those experiencing mild avoidance of specific situations (such as crowded stores or public transport) to those with more severe, isolating avoidance patterns. The application is intended to be used under a gradual, self-paced exposure model, allowing patients to progress at a difficulty level appropriate to their current comfort threshold.

Beyond direct patient use, the project is also relevant to mental health professionals — such as therapists and clinicians specializing in anxiety and exposure therapy — who may use the tool to support treatment plans or as a supplementary resource alongside traditional therapy sessions. Additionally, the project serves as a proof-of-concept for researchers interested in the intersection of interactive simulation and clinical exposure therapy techniques.

3. Comparable Products / Approaches

A small number of existing software tools apply similar principles to exposure therapy for agoraphobia and related anxiety disorders, though most differ significantly from Serenity Shift's approach.

Agoraphobia Free, developed by Thrive Therapeutic Software, is a mobile CBT-based application that uses 3D characters and simulated real-life situations. It guides users through a virtual character experiencing agoraphobia, walking them through tasks based on cognitive restructuring and systematic desensitization rather than direct first-person exposure simulation. Unlike Serenity Shift, the user observes and guides a virtual character rather than directly navigating a simulated environment themselves. [PubMed Central](#)

SyMptOMS-ET is a geolocation-based smartphone application designed to guide patients through in-vivo exposure therapy by directing them to real-world locations relevant to their treatment rather than providing a simulated environment. This approach relies on the patient physically visiting real locations, which reintroduces the unpredictability and logistical difficulty that simulation-based tools like Serenity Shift are designed to avoid. [Sage Journals](#)

Compared to these approaches, Serenity Shift's distinguishing feature is a fully simulated, first-person, navigable city environment on standard PC hardware, with directly adjustable stress variables (NPC density, ambient sound, traffic noise) and structured difficulty progression — rather than a guided narrative, a real-world location tool, or a general-purpose CBT companion app.

4. System Requirements

4.1 Functional Requirements

- **City navigation** — the player can freely move through a realistic, simulated 3D city environment using standard keyboard and mouse controls.
- **Difficulty level selection** — the system provides four distinct difficulty levels (tutorial, easy, medium, hard) within the same city environment, each adjusting the intensity of stress-inducing stimuli rather than loading a separate scene.
- **Tutorial level** — an onboarding level that teaches the player core navigation and interaction controls, including moving toward a marked area/objective, before exposure-based gameplay begins.
- **NPC-catching system** — players can identify and catch specifically flagged NPCs as part of mission objectives, with catchable NPCs distinguished from regular ambient pedestrians.
- **Bird-catching mechanic** — players can locate and interact with birds placed throughout the environment as part of mission objectives.
- **"Django" — finding the street performer** — players must locate a street-performer character positioned somewhere within the city environment.
- **"Django" — musical-note riddle** — once found, players complete an interactive musical-note-sequence puzzle presented by Django, supported by spatial audio cues.
- **Toast notification system** — the player receives an encouraging toast message (with a chime sound) each time they find an NPC or bird objective, in the medium and hard difficulty levels. Additionally, every level (tutorial, easy, medium, hard) displays a separate completion toast once its main goal is achieved (e.g., solving Django's note sequence, reaching the target in the tutorial, or finding all required NPCs/birds), showing the total time taken — regardless of whether the player has toggled the timer display off.
- **NPC system** — non-player characters populate the city streets, with density varying by difficulty level.
- **Car/traffic system** — vehicles move along city streets exclusively at the hard difficulty level, adding traffic and honking sound cues as an additional stressor at the highest intensity tier.

- **Bird-flying system** — birds fly freely throughout the sky above the city as part of the environment's ambient life, separate from the ground-level bird-catching mechanic.
- **Hidden gardens** — calm, aesthetically distinct garden areas are scattered throughout the city for the player to discover, giving them an additional reason to continue exploring after completing missions.
- **Authentication and progress saving** — players can create an account and have their progress saved and retrieved via cloud-based storage.
- **Pause menu** — players can pause the game at any point, with toggle options for volume, timer, and compass display.
- **Compass navigation aid** — an on-screen compass points the player toward their current objective.
- **Timer and performance tracking** — the system tracks completion time and mission performance for post-session review.
- **Objective particle system** — a particle effect is displayed at each objective's location, giving the player a visual marker to guide them, complementing the compass and 3D audio cues to prevent them from getting lost in the city.
- **Varied ambient sound per difficulty level** — the city's ambient soundscape (crowd noise, general city sound) changes intensity across difficulty levels and can be toggled on/off independently via the pause menu.

4.2 Non-Functional Requirements

- **Performance** — the game maintains a stable frame rate during navigation, NPC interaction, and audio playback, with no noticeable latency in controller/input response.
- **Build size and web accessibility** — the WebGL build is optimized to approximately 150MB, supporting full-screen display across various screens and platforms, including curved monitors.
- **Usability** — controls and menus are intuitive enough for users with no prior gaming experience, given the therapeutic (non-gamer) target audience.
- **Session control and patient safety** — the player can pause or exit the experience at any point, ensuring the patient always retains control over their exposure session.
- **Persistent accessibility controls** — toggles for stimuli-related settings (volume, ambient sound, timer, etc.) are shared and persistent across all difficulty levels; disabling a setting at one difficulty (e.g., medium) keeps it disabled across the others (easy, hard, and tutorial) until re-enabled.
- **Data persistence** — player progress and session data are reliably saved and retrievable via cloud-based storage in the PC version; the WebGL build does not include cloud-based persistence.
- **Platform compatibility** — the game runs on standard consumer-grade PC hardware without requiring specialized equipment.

5. Architecture & Design

5.1 System Architecture

Serenity Shift PC is built as a single-player, client-side application in Unity, structured around a single shared city scene that hosts all difficulty levels. Rather than loading separate environments per difficulty, the system dynamically adjusts stressor intensity — NPC density, ambient sound levels, and car/traffic presence — within the same scene based on the difficulty selected by the player.

At a high level, the architecture consists of the following core modules:

- **Player Controller** — handles movement, camera control, and interaction input (keyboard/mouse).
- **Difficulty Manager** — governs which stressor parameters (NPC count, ambient sound intensity, traffic presence) are active for the currently selected difficulty level, applied within the shared scene, and adjusts the complexity of mission objectives accordingly for each difficulty tier.
- **NPC System** — manages pedestrian spawning, pathfinding (NavMesh-based), and the subset of NPCs flagged as catchable for mission objectives.

Environmental Systems — includes the bird-flying system (ambient sky life), the car/traffic system (active only at hard difficulty), the city's ambient soundscape, and hidden gardens scattered throughout the city. The gardens serve as optional points of interest that give the player a reason to keep exploring the city after completing their missions, offering a calm, aesthetically pleasing space distinct from the surrounding stressors.

- **Mission/Objective System** — tracks and manages player objectives (NPC catching, bird catching, and the two-part "Django" encounter). Its primary purpose is to divert the player's attention while they navigate an open public space — a scenario that is inherently stressful for individuals with agoraphobia — by giving them a sense of accomplishment and capability within that stressful activity.
- **UI System** — includes the pause menu (with persistent toggles for volume, ambient sound, and timer), the compass navigation aid, and timer/performance display.
- **Authentication & Save System** — (PC build only) handles account authentication and cloud-based progress saving; unavailable in the WebGL build.

5.2 Technologies Used

Unity 6 (6000.2.8f1) — primary game engine used for development.

C# — scripting language used for all gameplay logic, systems, and UI behavior.

Unity Gaming Services (Authentication & CloudSave) — used for account authentication and persistent progress saving in the PC build.

WebGL build pipeline — used to export a browser-playable version of the game, optimized for size (~150MB) and full-screen compatibility across various screens, including curved monitors.

Git / GitHub — used for version control and repository management (Rehab-Game-Dev organization).

Continuous integration / code linting — a GitHub Actions workflow runs `dotnet-format` in check mode against all C# scripts in `Assets/Scripts` on every push and pull request, enforcing consistent code style and formatting.

5.3 Scene & Level Structure

tent city scene shared across all four difficulty tiers (tutorial, easy, medium, hard). This design choice reflects the project's therapeutic goal: presenting the player with one consistent, learnable environment, while varying only the intensity of stimuli within it as difficulty increases.

The city environment itself was built using a modular city-construction asset, populated with streets, buildings, NPC pathing routes, and points of interest such as the hidden gardens and the "Django" street-performer encounter. Within this single scene, the Difficulty Manager dynamically adjusts:

- **NPC density and behavior** — increasing the number and complexity of pedestrian interactions at higher difficulty levels.
- **Ambient sound intensity** — from calm, minimal city ambience at tutorial/easy, to louder, denser crowd and city noise at medium/hard.
- **Traffic presence** — cars and honking sounds are introduced exclusively at the hard difficulty level.
- **Mission complexity** — objectives (NPC catching, bird catching, the Django puzzle) scale in difficulty alongside the environment.

This shared-scene approach also simplifies technical maintenance and ensures visual and spatial consistency for the player as they progress — they are not learning a new map at each stage, only adapting to an increasingly demanding version of the same one.

6. Development Process

6.1 Timeline / Chronological Overview

Development of Serenity Shift PC followed a gradual, feature-by-feature progression, building up the environment and gameplay systems before layering in polish and optimization.

Work began with constructing the **city environment**, establishing the streets, buildings, and overall layout that would serve as the backdrop for every subsequent feature. Once the environment was in place, the **player character and navigation controls** were implemented, allowing free movement through the city.

With core movement established, the **NPC system** was introduced, followed by the **bird-flying system**, adding ambient life to both the streets and sky. Mission design then began

with the **easy mission**, followed by the **medium mission**, after which the **tutorial level** was built to onboard new players using the mechanics already in place.

Next, **cloud-based authentication** was integrated to support account creation and progress saving, followed by the **car/traffic system**, and finally the **hard mission**, completing the four-tier difficulty structure. At this stage, the project was presented at **Expo TLV**.

Following the Expo presentation, development shifted toward refinement: the **hidden gardens** were added, along with a round of **bug fixes**, and additional quality-of-life features such as the **full-featured pause menu** (volume, ambient sound, and timer toggles) and a **level-loading bar**. Development concluded with the introduction of **CI linting** via GitHub Actions, followed by **WebGL build optimization**, reducing the build size for smoother web deployment.

6.2 Technical Challenges & Solutions

NPC Spawning and Wall Clipping

Early implementations of the NPC system occasionally spawned pedestrians inside walls or obstacles, breaking immersion. This was resolved with clearance checks in the NPC spawner, ensuring NPCs only spawn in valid, obstacle-free positions before entering the scene.

Pause Menu State Persistence Across Scenes

Ensuring that pause menu toggles (volume, ambient sound, timer) remained consistent across all difficulty levels — rather than resetting when the player switched between them — required persisting UI state independently of the shared scene's lifecycle, rather than relying on default per-scene state.

Difficulty-Based Mission Scaling

Designing how mission complexity should scale across the four difficulty levels was guided by occupational therapy students, ensuring the progression in stressor intensity and task difficulty aligned with therapeutic exposure principles rather than purely gameplay-driven difficulty curves.

Integrating Diverse City and Garden Assets

Building a cohesive, well-structured city environment — including the hidden gardens — required integrating assets from different sources while maintaining smooth, consistent 3D animation throughout. This posed a challenge because assets not originally designed to work together needed to be blended into a single coherent, animated environment.

Django Model and Guitar Clipping

The Django character required an animated character model combined with a separate guitar model, attached together in a way that avoided z-fighting and mesh clipping between the two. This was solved through creative positioning and attachment approaches to keep the guitar visually locked to the character's animation without visual artifacts.

Distance-Based 3D Audio for Objective Tracking

Beyond general ambience, 3D audio needed to serve a functional purpose: helping players track and locate objectives such as the street performer and the birds. This required carefully

tuning volume falloff and spatial audio parameters so that sound intensity increased noticeably as the player approached a target, giving them an audible cue to guide navigation — without the volume curve being either too subtle to notice or too jarring as they got close.

Navigation and Accessibility in a Large City

Given the scale of the city environment, players risked losing track of their objectives or feeling disoriented while exploring. This was addressed by combining the compass — which points toward the player's nearest objective — with a particle effect placed at each objective itself, giving players two complementary visual cues to guide them and preventing them from getting lost in the open city environment.

WebGL Build Size

The initial WebGL build size was significantly larger than needed for smooth web deployment. Through texture compression and audio compression, the build size was reduced from roughly 365MB down to approximately 150MB, improving load times and accessibility across platforms, including various screen types and curved monitors.

7. Key Features

7.1 Authentication & Progress Saving

Serenity Shift PC includes a full authentication system, allowing players to create an account and log in before starting a session. Players who prefer not to create an account can also play as a guest. Once authenticated, player progress is saved via cloud-based storage, allowing sessions to be resumed and performance to be tracked over time. This feature is available exclusively in the standalone PC build; the WebGL version does not include cloud-based authentication or save functionality.

7.2 NPC & Bird-Catching Mechanics

Serenity Shift PC populates its city environment with non-player characters (NPCs) that behave as realistic pedestrians, using NavMesh-based pathfinding to move naturally through the streets. Their behavior remains consistent across all difficulty levels; only their density (the number of NPCs present) scales with difficulty, contributing to the overall stress intensity of each session.

Among these NPCs, a select subset is flagged as catchable, forming the basis of the NPC-catching mission. Players must identify and interact with these specific NPCs, distinguishing them from the general crowd of ambient pedestrians — a task that requires sustained attention and active engagement with the busy street environment rather than passive avoidance of it.

A parallel mechanic, the bird-catching mission, follows a similar principle at a different layer of the environment. Each session, three birds are randomly selected from a pool of fifteen predetermined spawn locations throughout the city, keeping the objective's location fresh across playthroughs and supporting replayability. These mission birds use a unique model, visually distinct from both the birds freely flying in the sky as ambient life and the birds found in the hidden gardens, making them clearly identifiable as objectives.

To help players locate both bird and NPC objectives within the large city environment, each is highlighted with yellow particle effects, and the compass points toward the nearest one at any given time — giving players two complementary cues so they can search purposefully without becoming lost or disoriented.

Together, these mechanics serve the project's core therapeutic goal: keeping the player actively engaged and purposefully occupied within an open public space, redirecting attention away from anxiety-inducing stimuli through structured, achievable tasks.

7.3 The “Django” Puzzle

Serenity Shift PC features a distinct two-part encounter centered on Django, a street-performer character placed within the city environment, serving as the hard-difficulty mission's core objective.

Locating Django draws on the same navigational aids used elsewhere in the game — the compass points toward him, and faint musical-note particles emanate from him, visible from a distance. However, this mission is intentionally designed to emphasize audio guidance above the other two cues: the sound of his music grows louder as players approach and quieter as they move away, encouraging players to actively navigate the open street environment by ear rather than relying solely on visual aids.

Once Django is found, he announces that there is a riddle nearby and tells the player a sequence of colors. The mission objective then updates to display a set of written musical notes, each paired with the corresponding color Django specified. The colorful note-blocks needed to solve the riddle — each producing one of the notes of the solfège scale (do, re, mi, fa, sol, la, si) when interacted with — are all located near Django himself. Players must find and press them in the correct order to reproduce the note sequence matching the colors and notes given in the mission objective.

After the riddle is successfully solved, Django's music continues playing at 50% volume for the remainder of the session. This allows players who choose to keep roaming the city afterward to still enjoy the ambience of his performance, without the audio being overwhelming as they move on to explore other parts of the environment, such as the hidden gardens.

7.4 Audio Systems

Serenity Shift PC's city environment is built around a layered system of ambient elements, each contributing to the overall sense of atmosphere and, more importantly, to the controlled scaling of stress intensity across difficulty levels.

The city's ambient soundscape — general crowd and city noise — varies in intensity across difficulty levels, growing louder and busier at higher difficulties and remaining calmer and quieter at lower ones. This ambient sound, along with other audio elements such as volume and the timer, can be independently toggled on or off via the pause menu, with these preferences persisting across all difficulty levels rather than resetting between sessions.

At the hard difficulty level specifically, a car and traffic system is introduced, adding moving vehicles and honking sounds as an additional environmental stressor exclusive to the highest intensity tier.

Beyond ambient sound, birds populate the sky as part of the environment's ambient life, flying freely and independently of the ground-level bird-catching mission objectives. Scattered throughout the city are also hidden gardens — calm, visually distinct areas that give players an incentive to keep exploring after completing their missions, offering a relaxing counterpoint to the surrounding urban stressors.

Spatial 3D audio underlies much of this system, most notably in guiding players toward objectives: sound cues grow louder as players approach a target (such as Django) and fade as they move away, turning audio into a functional navigational tool rather than pure ambience.

In the easy and medium difficulty levels, successfully finding an individual objective (an NPC or a bird) is accompanied by a chime sound and an encouraging toast message, providing immediate positive reinforcement. Separately, every level displays a completion toast once its main goal is achieved (such as completing Django's note sequence, reaching the target in the tutorial, or finding all required NPCs or birds), reporting the total time taken regardless of whether the timer display was toggled off.

7.5 Navigation & Accessibility Aids

Given the scale of Serenity Shift PC's city environment, a set of complementary aids was designed to help players navigate confidently without becoming lost or overwhelmed — an important consideration for a therapeutic tool meant to build a sense of capability rather than frustration.

A compass is displayed on-screen throughout gameplay, consistently pointing toward the player's nearest active objective, whether an NPC, a bird, or Django. Alongside it, each objective is marked with a particle effect — colored yellow for NPCs and birds — visible to the player as they approach, serving as a secondary visual cue independent of the compass.

These aids work together with the game's spatial audio system, which reinforces proximity through sound. In the medium difficulty level's bird-finding mission, players hear a bird's chirp grow louder as they get closer to it, mirroring the same proximity-based audio principle used in the search for Django, where sound guidance is the primary means of locating him, supplemented by faint musical-note particles and the compass.

Together, the compass, particle effects, and spatial audio form a layered accessibility system, ensuring players — regardless of their familiarity with games or their level of anxiety in a large open environment — can always find their way toward their next objective within the city.

8. Testing & Evaluation

8.1 Functional Testing

Beyond manual playtesting during development, Serenity Shift PC's codebase includes an automated unit testing suite built with Unity's Test Framework.

To support this, the project's structure was reorganized into two separate assembly definitions: `SerenityShift.asmdef` for production scripts (previously compiling into Unity's implicit default assembly, which is not directly referenceable by test code), and a dedicated `SerenityShift.EditModeTests.asmdef` for the Edit Mode test assembly. Two targeted behavior-preserving refactors were also made to expose previously private logic for testing: `TimerManager.cs` had its duplicated time-formatting logic extracted into a public static `FormatTime(float)` method, and `SequenceChecker.cs` had its prefix-matching logic extracted into a public static `IsSequencePrefixMatch(...)` method, alongside a new public `IsPuzzleSolved` getter.

In total, 23 unit tests were written across five test files:

- **TimerManagerTests** (5 tests) — covering time-formatting edge cases, including zero values, digit padding, minute rollover, and floor-vs-round behavior.
- **SequenceCheckerTests** (8 tests) — covering the Django puzzle's prefix-matching logic, including instance-level tests through the actual `NotePressed()` method.
- **GameManagerTests** (4 tests) — covering NPC-catch counting and the mission-completion threshold transition.
- **BirdGameManagerTests** (3 tests) — covering bird-catch counting logic.
- **BirdManagerTests** (3 tests) — covering random bird-pool activation, including the edge case of a pool smaller than the requested amount.

All 23 tests were run in Unity's Test Runner and confirmed passing (23/23, 0 failures).

In addition to unit testing, code quality was enforced through continuous integration: a GitHub Actions workflow runs `dotnet-format` in check mode against all C# scripts in `Assets/Scripts` on every push and pull request, automatically flagging formatting and style inconsistencies before they are merged.

8.2 Clinical Feedback

Separately from the occupational therapy students who earlier guided the game's therapeutic design (see Section 6.2), player reception was evaluated through a dedicated feedback survey. The survey was built in Hebrew using Google Forms and distributed to individuals who tried the game. Respondents were asked to evaluate the game from a professional, therapeutic-impression standpoint, rating six aspects on a 1–5 scale, alongside questions on where in a treatment process they would consider integrating the tool, how likely they would be to recommend it, and an overall rating, plus optional free-text feedback. A total of 17 responses were collected.

Average ratings for the 17 respondents were as follows:

Aspect	Average (out of 5)
Environment size appropriate for treatment	4.4
Realism of the experience	4.2

Aspect	Average (out of 5)
Clarity of missions	4.0
Comfort/naturalness of controls	4.4
Difficulty/pace suitability for the patient population	4.1
Scenes' fidelity to real agoraphobia triggers	3.9
Likelihood to recommend to a friend with agoraphobia	4.3
Overall rating	4.7

On where in a treatment process the tool would be used, most respondents (12 of 17) said they would use it **alongside ongoing therapy**, 4 said they would use it **before real-world exposure**, and 1 said they would not integrate it.

Recurring themes in the free-text feedback included:

- Multiple respondents described the exposure experience as controlled and gradual; one noted it felt slightly overwhelming at first but became appropriate as the session progressed.
- The most common suggestion for improvement was polishing the graphics and UI.
- Several respondents suggested expanding the game with additional levels or scenarios tailored to other mental health conditions beyond agoraphobia.
- One respondent specifically recommended introducing visual aids (such as the compass or particle cues) more gradually, to create a softer initial experience.

8.3 Public Presentation

Serenity Shift was presented at Expo TLV, Ariel University's annual competition for students of the "Computer Game Development" course, in which participants showcase original games developed during the course to judges from leading companies in the Israeli gaming industry. The competition gives students the opportunity to present their work to industry professionals and gain firsthand exposure to the professional game development world.

At the event, Serenity Shift was demonstrated to an audience of industry game developers and businesspeople. Overall impressions were positive, with encouraging feedback received from several game developers in attendance, and one particularly enthusiastic response from a company CEO present at the exhibition.

9. Results & Conclusions

Serenity Shift PC successfully delivers on its core goal: a controlled, gradual, and engaging exposure simulation for individuals with agoraphobia, built entirely around principles that are difficult to replicate in real-world exposure sessions. The game offers a fully navigable city environment with four distinct difficulty levels, each precisely scaling stressor intensity — NPC density, ambient sound, and traffic — while keeping mission complexity aligned with therapeutic exposure principles, guided by occupational therapy students during development.

Beyond the core exposure mechanic, the project layers in features designed specifically to support the player's experience: attention-diverting missions (NPC catching, bird catching,

and the Django encounter), accessibility aids (compass, particle cues, and distance-based spatial audio) to prevent disorientation in a large open environment, and calming points of interest such as the hidden gardens that reward exploration beyond the missions themselves.

The project's technical foundation reflects genuine engineering rigor for a project: a persistent, difficulty-independent scene architecture, an automated unit testing suite (23 tests across five files) validating core gameplay logic, continuous integration via linting, and a WebGL build optimized down to approximately 150MB for broad web accessibility.

Feedback gathered from 17 respondents to the player survey was strongly positive, with an average overall rating of 4.7 out of 5 and the majority indicating they would use the tool alongside ongoing therapy. This was reinforced by the positive reception at Expo TLV, where the game was well received by industry game developers and businesspeople alike.

Taken together, these results support the project's central premise: that a PC-based simulation, built with deliberate control over environmental stressors and guided by therapeutic design principles, can offer a viable, accessible, and engaging complement to traditional in-person exposure therapy for agoraphobia.

10. Future Work

Visual polish — improving graphics and UI fidelity, the most frequently requested improvement in player feedback.

Gradual introduction of accessibility aids — easing players into visual guidance elements (compass, particle cues) more gradually, rather than presenting them at full strength from the start, to create a softer onboarding experience.

Train station environment — building a new, currently unimplemented train station environment with its own set of missions, expanding the game beyond the current city scene into a new exposure scenario.

Additional difficulty levels or scenarios — further expanding beyond the current four-tier structure with additional levels or environments (e.g., supermarket, public transport), as suggested by survey respondents.

Expanded therapeutic scope — adapting or extending the simulation framework to support exposure scenarios for other anxiety-related conditions beyond agoraphobia, as suggested by several respondents.

Formal clinical evaluation — conducting a structured evaluation with therapists and patients directly, building on the informal feedback gathered so far, to validate therapeutic effectiveness more rigorously.

Therapist observation mode — a dedicated mode allowing a therapist to observe or monitor a patient's session in real time, supporting guided or supervised use of the tool.

11. References

González-Pérez, A., Diaz-Sanahuja, L., Matey-Sanz, M., Osma, J., Granell, C., Bretón-López, J., & Casteleyn, S. (2024). Towards a self-applied, mobile-based geolocated exposure therapy software for anxiety disorders: SyMptOMS-ET app. *Digital Health*, 10. <https://doi.org/10.1177/20552076241283942>

Guth, M., Wiebe, A., Ekhlas-Schumann, M., Rodenjohann, A., Rohlfen, F., Buchholz, J., Selaskowski, B., Philipsen, A., & Braun, N. (2025). Mobile app-guided exposure therapy for panic disorder with and without agoraphobia: Randomized controlled trial. *Journal of Medical Internet Research*, 27, e76389. <https://doi.org/10.2196/76389>

12. Appendices

Appendix A: Selected Code Samples

This appendix presents a set of representative code excerpts from the SerenityShiftPC project, selected to illustrate the range of engineering concerns addressed during development: asynchronous programming, cloud-service integration, algorithm design, randomization, cross-system state management, procedural content placement, reusable component design, and automated testing.

12.A.1 Asynchronous Scene Loading with a Smoothed Progress Bar

```
private IEnumerator LoadSceneAsync(string sceneName)
{
    if (loadingPanel != null) loadingPanel.SetActive(true);
    if (progressBar != null) progressBar.value = 0f;

    foreach (GameObject element in menuElementsToHide)
    {
        if (element != null) element.SetActive(false);
    }

    yield return null; // let the engine render the loading panel before the
    load call blocks the thread

    AsyncOperation operation = SceneManager.LoadSceneAsync(sceneName);
    operation.allowSceneActivation = false;

    float displayedProgress = 0f;

    while (!operation.isDone)
    {
        float targetProgress = Mathf.Clamp01(operation.progress / 0.9f);
        displayedProgress = Mathf.MoveTowards(displayedProgress,
        targetProgress, Time.deltaTime * progressFillSpeed);

        if (progressBar != null) progressBar.value = displayedProgress;

        if (operation.progress >= 0.9f && displayedProgress >= 0.999f)
        {
            operation.allowSceneActivation = true;
        }

        yield return null;
    }
}
```

This method loads a game level in the background while driving a UI progress bar, and it solves two real problems rather than being a naive wrapper around Unity's API. First, Unity's `AsyncOperation.progress` only ever reports values between 0 and 0.9 while a scene is loading — the final 0.1 is reserved for the scene-activation step — so the raw value is rescaled (`progress / 0.9f`) to give the player a bar that reads as a clean 0-100%.

Second, when a scene is already cached (as often happens inside the Unity Editor), the load can complete in a single frame, which would make the progress bar jump instantly from empty to full — visually unconvincing and inconsistent with how the bar behaves on a slower load. `Mathf.MoveTowards` is used to animate the displayed value toward the real value at a capped rate, so the bar always reads as gradual motion regardless of how fast the underlying asset load actually is.

Finally, `operation.allowSceneActivation` is deliberately held at `false` until both the load is finished and the bar has visually caught up to 100%, guaranteeing the player never sees the scene switch before the loading UI has finished communicating that it is ready.

12.A.2. Cloud-Backed Login with Resume-From-Last-Level

```
private async void LoadLevelData()
{
    var data = await CloudSaveService.Instance.Data.Player.LoadAsync(new
    HashSet<string> { "saved_level" });

    if (data.ContainsKey("saved_level"))
    {
        int levelToLoad = data["saved_level"].Value.GetAs<int>();

        switch (levelToLoad)
        {
            case 1: SceneManager.LoadScene("StreetScene - tutorial"); break;
            case 2: SceneManager.LoadScene("StreetScene - easy"); break;
            case 3: SceneManager.LoadScene("StreetScene - medium"); break;
            case 4: SceneManager.LoadScene("StreetScene - hard"); break;
            default: SceneManager.LoadScene("MainMenu"); break;
        }
    }
    else
    {
        SceneManager.LoadScene("StreetScene - tutorial");
    }
}

// Paired with, in the level-selection menu script:

private async void SaveCurrentLevel(int level)
{
    if (!AuthenticationService.Instance.IsSignedIn) return;

    var data = new Dictionary<string, object> { { "saved_level", level } };
    await CloudSaveService.Instance.Data.Player.SaveAsync(data);
}
```

Together, these two methods implement persistent player progress using Unity's Cloud Save service. The design choice worth highlighting is where the saving happens: rather than writing progress only when a level is completed, `SaveCurrentLevel` is called the moment a player selects a level from the menu, so a returning player resumes wherever they last were, not wherever they last finished.

`LoadLevelData` is written defensively for two edge cases that are easy to overlook: a genuinely new account (no `saved_level` key exists yet, falls back to the tutorial) and a corrupted or unexpected value (the default case in the switch also falls back safely rather than crashing).

The `IsSignedIn` guard in `SaveCurrentLevel` reflects a second design consideration: the game supports a guest login path with no cloud account behind it, and calling the Cloud Save API without an

authenticated session would throw an exception — so progress is silently not persisted for guests rather than the game failing.

12.A.3. A Testable Algorithm Extracted from Game Logic (Note-Sequence Puzzle)

```
public static bool IsSequencePrefixMatch(IReadOnlyList<string> played,
IReadOnlyList<string> correct)
{
    if (played.Count > correct.Count) return false;

    for (int i = 0; i < played.Count; i++)
    {
        if (played[i] != correct[i])
        {
            return false;
        }
    }
    return true;
}
```

This method is the core of an in-game musical puzzle in which the player must reproduce a sequence of notes in the correct order, and it is checked after every note rather than only once the full sequence has been entered — so a mistake is caught immediately instead of after eight inputs.

What makes it worth including here is not the algorithm's complexity (it is a straightforward prefix comparison) but its shape: it was deliberately extracted out of the `MonoBehaviour` that originally called it and rewritten as a static method operating only on its parameters, with no dependency on Unity's scene state, `GameObjects`, or instance fields. That refactor is what makes the method unit-testable in isolation - a change that cost nothing in terms of behavior but turned an untestable piece of gameplay logic into one that can be verified automatically.

12.A.4. Weighted Random Selection Without Replacement

```
void ActivateRandomBirds()
{
    List<GameObject> availableBirds = new List<GameObject>(birdsInScene);

    for (int i = 0; i < amountToSpawn; i++)
    {
        if (availableBirds.Count == 0) break;

        int randomIndex = Random.Range(0, availableBirds.Count);
        GameObject selectedBird = availableBirds[randomIndex];
        selectedBird.SetActive(true);

        availableBirds.RemoveAt(randomIndex);
    }
}
```

This method selects a random subset of a fixed pool of pre-placed bird objects to activate as catchable targets for a given level. The algorithm is a simple form of sampling without replacement: rather than repeatedly picking a random index and risking the same object being chosen twice, each selected element is removed from a working copy of the list (`availableBirds`) before the next pick, guaranteeing no duplicates.

The method also handles the case where the requested amount exceeds the pool size correctly — the loop simply exits early via the break once the pool is exhausted, rather than throwing an index-out-of-range exception. This method is directly covered by unit tests verifying both the exact-count case and the pool-smaller-than-requested edge case.

12.A.5. Centralized Pause State Across Independent Systems

```
public void Pause()
{
    isPaused = true;
    Time.timeScale = 0f;
    pauseOverlay.SetActive(true);
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;

    foreach (NavMeshAgent agent in
FindObjectsByType<NavMeshAgent>(FindObjectsSortMode.None))
    {
        agent.isStopped = true;
    }

    pausedAudioSources.Clear();
    AudioSource[] allAudioSources =
FindObjectsByType<AudioSource>(FindObjectsSortMode.None);
    foreach (AudioSource audio in allAudioSources)
    {
        if (audio.isPlaying)
        {
            audio.Pause();
            pausedAudioSources.Add(audio);
        }
    }
}
```

Pausing a real-time 3D game is not a single operation but a coordination problem across several independent systems that were not designed with pausing in mind — physics-driven navigation, audio playback, and time-dependent gameplay logic all need to be told to stop simultaneously.

This method addresses that by using Unity's global `Time.timeScale = 0f` for time-based systems (which automatically freezes anything driven by `Time.deltaTime`), while explicitly and separately halting every active `NavMeshAgent` in the scene (agents ignore `timeScale` and would otherwise continue moving) and every currently-playing `AudioSource`.

The `pausedAudioSources` list is a deliberate design detail: rather than pausing and later resuming every audio source indiscriminately, only sources that were actually playing at the moment of pausing are recorded, so `Resume()` does not incorrectly start audio that was already silent beforehand.

12.A.6. A Unit Test for the Extracted Puzzle Logic

```
[Test]
public void IsSequencePrefixMatch_WrongNoteMidway_ReturnsFalse()
{
    var played = new List<string> { "Do", "Mi", "Re" }; // third note should be
    "Fa"
    Assert.IsFalse(SequenceChecker.IsSequencePrefixMatch(played,
CorrectSequence));
}
```

```
[Test]
public void IsSequencePrefixMatch_PlayedLongerThanCorrect_ReturnsFalse()
{
    var played = new List<string> { "Do", "Mi", "Fa", "Sol", "Fa", "Mi", "Re",
    "Do", "Do" };
    Assert.IsFalse(SequenceChecker.IsSequencePrefixMatch(played,
    CorrectSequence));
}
```

These are two of a small suite of automated tests written against the method in section 3, using Unity's Test Framework (built on NUnit). They illustrate a standard unit-testing practice: rather than only testing the happy path (a correct sequence), tests specifically target boundary and failure conditions — a wrong note appearing partway through, and an input longer than the target sequence could ever validly be.

Writing these tests required a small, behavior-preserving refactor of the original class (extracting the static method shown in section 3, and adding an assembly definition so the project's code compiled separately from Unity's implicit default assembly, allowing a dedicated test assembly to reference it) — a concrete example of how testability is often not free, but can be achieved with minimal, low-risk structural changes to existing code rather than a rewrite.

12.A.7. NPC Spawning with NavMesh Validation and Obstacle Rejection

```
[ContextMenu("Spawn NPCs Now")]
public void SpawnNPCs()
{
    GameObject parentGroup = new GameObject("Generated_NPCs");
    parentGroup.transform.position = transform.position;

    int spawned = 0;
    int attempts = 0;
    int maxAttempts = amount * 10;

    while (spawned < amount && attempts < maxAttempts)
    {
        attempts++;
        Vector3 randomPoint = transform.position + Random.insideUnitSphere *
range;

        NavMeshHit hit;
        if (!NavMesh.SamplePosition(randomPoint, out hit, 10.0f,
NavMesh.AllAreas))
            continue;

        Vector3 checkLow = hit.position + Vector3.up * 0.5f;
        Vector3 checkMid = hit.position + Vector3.up * 1.5f;

        if (Physics.CheckSphere(checkLow, clearanceRadius, buildingLayer) ||
Physics.CheckSphere(checkMid, clearanceRadius, buildingLayer))
            continue;

        GameObject newNPC = Instantiate(
            npcPrefabs[Random.Range(0, npcPrefabs.Length)],
            hit.position,
            Quaternion.Euler(0, Random.Range(0, 360), 0)
        );
        newNPC.transform.localScale = spawnScale;
        newNPC.transform.parent = parentGroup.transform;
        spawned++;
    }
}
```

```

    if (spawned < amount)
        Debug.LogWarning($"NPCSpawner: Only spawned {spawned}/{amount} NPCs.");
}

```

Populating a street scene with background characters at valid, non-overlapping positions is a harder problem than picking random points, because a random point inside a bounding sphere has no guarantee of being walkable ground rather than the inside of a building.

This method solves that with a two-stage validation per candidate point: `NavMesh.SamplePosition` first snaps the random point onto the nearest walkable surface (rejecting it outright if nothing walkable is nearby), and `Physics.CheckSphere` then confirms the snapped point is not physically obstructed — checked at two different heights so that low walls and low overhangs are both caught, since an NPC could otherwise be spawned inside geometry that the NavMesh alone does not account for.

The `attempts/maxAttempts` bound is a defensive engineering detail: without it, a scene that is mostly unwalkable (or requests far more NPCs than the space can hold) would send this method into an infinite loop; instead it gives up gracefully and logs how many it actually managed to place.

12.A.8. Bird-Catching Mechanic with Multi-Controller Fallback

```

public void CatchBird()
{
    if (hasBeenCaught) return;
    hasBeenCaught = true;

    if (catchSound != null)
    {
        GameObject player = GameObject.FindGameObjectWithTag("Player");
        if (player != null)
        {
            AudioSource.PlayClipAtPoint(catchSound, player.transform.position);
        }
    }

    // Different levels use different message controllers, so try each in turn.
    if (GoalMessageController.Instance != null)
    {
        GoalMessageController.Instance.OnBirdCaught();
    }
    else if (BirdMessageController.Instance != null)
    {
        BirdMessageController.Instance.OnBirdCaught();
    }

    if (BirdGameManager.Instance != null)
    {
        BirdGameManager.Instance.BirdCaught();
    }
    else
    {
        Debug.LogError("BirdGameManager.Instance is NULL!");
    }

    Destroy(gameObject);
}

```

This method is attached identically to every catchable bird across three different level types, yet those levels do not all use the same UI/progress-tracking system — one level tracks only birds, another

tracks NPCs and birds together, and which controller is currently active is a scene-level configuration decision rather than something this script can know in advance.

Rather than hard-coding a single dependency, `CatchBird` checks for each possible controller's singleton Instance and calls whichever one is actually present in the loaded scene, falling through the chain until it finds one. This lets a single reusable component work correctly across every level without needing a level-specific variant of the script.

The `hasBeenCaught` flag guarding the top of the method is a simple idempotency check: because collision/trigger events can occasionally fire more than once for the same interaction in a single frame, the guard ensures the catch is only ever processed once, preventing the score from being incremented twice for one bird.

12.A.9. Traffic AI: Waypoint Navigation with Sensor-Based Obstacle Avoidance

```
void Update()
{
    if (!agent.isOnNavMesh) return;
    CheckForObstacles();
    if (isStopped) return;

    if (pathPoints == null || pathPoints.Count == 0) return;
    if (!agent.pathPending && agent.remainingDistance < 4f)
    {
        currentPointIndex++;
        if (currentPointIndex < pathPoints.Count)
        {
            agent.SetDestination(pathPoints[currentPointIndex].position);
        }
        else
        {
            Destroy(gameObject);
        }
    }
}

void CheckForObstacles()
{
    RaycastHit hit;
    Vector3 sensorStart = transform.position + transform.forward * 1.5f +
    Vector3.up * 0.5f;

    bool obstacleDetected = Physics.Raycast(sensorStart, transform.forward, out
    hit, detectionDistance, obstacleLayers)
    && hit.collider.gameObject != gameObject;

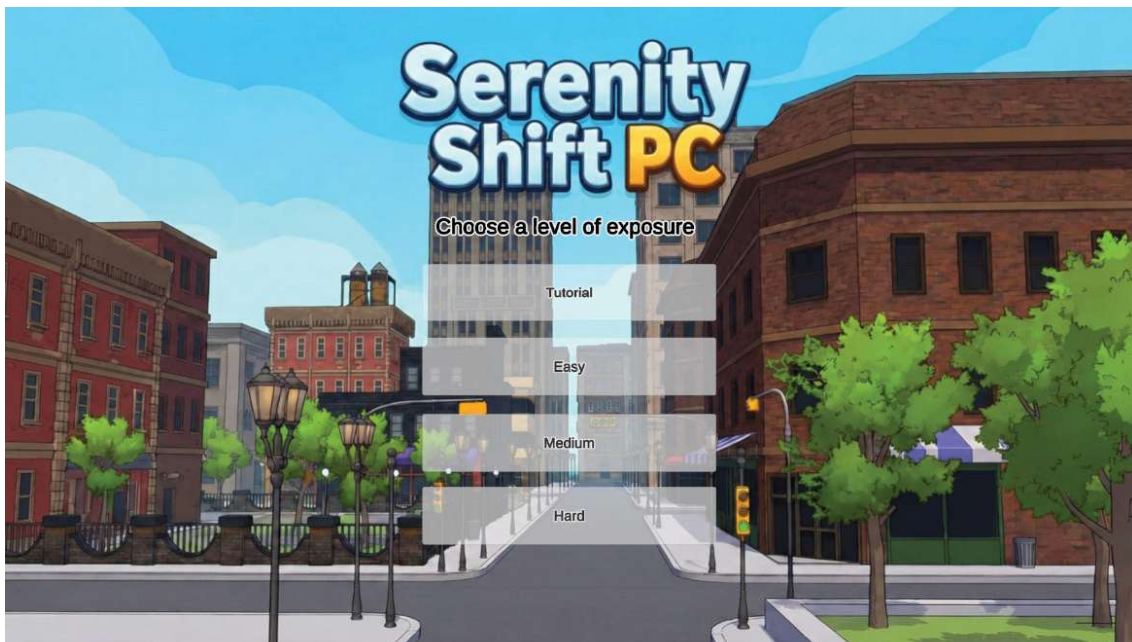
    if (obstacleDetected)
    {
        agent.isStopped = true;
        agent.velocity = Vector3.zero;
        agent.angularSpeed = 0f;
        isStopped = true;
    }
    else
    {
        agent.isStopped = false;
        agent.angularSpeed = 120f;
        isStopped = false;
    }
}
```

CarBrain combines two independent behaviors that are deliberately kept as separate responsibilities within the same Update loop: deterministic route-following (advancing through a fixed list of waypoints via `NavMeshAgent.SetDestination`) and reactive obstacle sensing (a forward-facing raycast "bumper" cast every frame).

This separation is what produces believable traffic behavior without needing a full traffic-light or intersection-management system — a car simply follows its assigned route unless its raycast sensor detects something ahead of it (typically another car, or the player), in which case it halts in place by zeroing the agent's velocity and steering rate, and automatically resumes the moment the sensor clears.

The `hit.collider.gameObject != gameObject` check exists because a car's own collider could otherwise register a hit against itself depending on collider geometry, which would cause it to permanently freeze. Reaching the final waypoint destroys the car rather than looping it, which is the intended behavior for through-traffic that should eventually leave the scene.

Appendix B: Screenshots



- Main menu



- Main street



- Hidden garden



- Django the street performer

Appendix C: Links

GitHub Repository: github.com/Rehab-Game-Dev/SerenityShiftPC

Playable Demo (itch.io): yarinkash1.itch.io/serenity-shift-pc-final-build-v1